

Computer Graphics Society

Sophomore Selections — WebX Task Round

Hello, Sophomores!

We hope you're all doing well!

We're thrilled to present the **Task Round of the Computer Graphics Society** for the **WebX Portfolio**.

Below, you'll find carefully curated problem statements across diverse domains. You're encouraged to choose **any one** of the tasks and give it your best shot. Not only will you enjoy the creative process, but you'll also gain hands-on experience in exciting and emerging areas of web development.

 **Remember — WebX is more than just traditional Web Development.**

List of Tasks (Attempt any one — or more, if you're up for it!)

1.  [Frontend & 3D](#)
2.  [Backend](#)
3.  [Blockchain](#)

Frontend & 3D Track: Interactive 3D Product Configurator

◆ Task

Design and develop a modern, **responsive** React-based web application featuring an **interactive 3D scene** using Three.js (preferably via React Three Fiber). Demonstrate your ability to seamlessly synchronize standard React UI state with a WebGL canvas, manage 3D assets, and build a deployment-ready UI.

Objectives

Build a "Product Configurator" (e.g., customizing a shoe, a car, or a piece of furniture) where users can interact with a 3D model (you can either create your custom 3D models or simply download it) through a React UI. It can be something as simple as having a bunch of options that one can combine to get multiple objects ("products") or something even better as per your creativity!

Evaluation will be done on your ability to load and optimize 3D assets, manage state between the DOM and the Canvas, implement 3D camera controls, and structure scalable frontend code. The first few points will be given higher precedence than the latter in evaluation.

Core Features

- **3D Scene & Asset Loading**
 - Initialize a 3D scene with appropriate lighting (ambient, directional, or environment maps) and a camera.
 - Load a real 3D model (use .gltf or .glb formats). You can use free models from platforms like Sketchfab or make your own custom ones.
 - Implement a Suspense fallback or Loading screen: Display a visually pleasing loading spinner or progress bar in the React UI while the 3D assets are being fetched and parsed over the network.
- **Interactive Controls (React to 3D)**
 - Develop a standard React UI (sidebar or bottom sheet) containing customization options.
 - Material Customization: Provide a color picker or preset color buttons in the UI that dynamically change the material color of specific meshes on the 3D model (e.g., changing the laces on a shoe, or the paint on a car).
 - Visibility Toggles: Add switches or checkboxes in the UI to show/hide specific parts of the 3D model (e.g., adding/removing a spoiler on a car model).
- **Canvas Interactions (3D to React)**
 - Implement OrbitControls so the user can easily rotate, pan, and zoom around the 3D object using their mouse or touch screen.

- Raycasting/Click Events: Allow users to click directly on specific parts of the 3D model. When a part is clicked, update the React UI to display the name of the selected part and highlight it in the menu.
- **Responsive Design**
 - Ensure the 3D canvas scales correctly when the browser window is resized without distorting the aspect ratio.
 - The UI panel must be fully responsive. On the desktop, it should sit neatly alongside the canvas. On mobile devices, it should collapse into a bottom drawer or an overlay menu so it doesn't block the 3D view.

Technical Requirements

- **Framework:** React (functional components and hooks).
- **3D Libraries:** three and @react-three/fiber (R3F is highly recommended for React integration). You may also use @react-three/drei for helpful abstractions like loaders and controls.
- **State Management:** Use a tool that plays nicely with both React and Three.js (e.g., Zustand is widely considered the best standard for R3F, but React Context API is also acceptable).
- **Styling:** Tailwind CSS, Chakra UI, Framer Motion (for UI animations), or standard CSS modules.
- **Performance:** Prevent unnecessary re-renders of the 3D canvas when React state updates. Ensure materials and geometries are handled efficiently.

Dockerization

- Dockerfile: Write a multi-stage Dockerfile to containerize your React + Three.js app:
- Build Stage: Use a Node.js base image, install dependencies, and build the production assets.
- Production Stage: Use an Nginx base image, copy the build output, and serve the static files via Nginx.
- Code Quality & Documentation
- Apply linting (ESLint) and formatting (Prettier).
- Write a concise README.md explaining your architectural choices (especially how you handled state between React and Three.js), setup instructions, and asset credits.

Bonus Points

Advanced 3D polish

- Post-Processing: Add visual flair using @react-three/postprocessing. Implement effects like subtle Bloom (glow), Ambient Occlusion (contact shadows), or Depth of Field.

- 3D HTML Annotations: Use Drei's `<Html>` component to anchor floating HTML labels to specific coordinates on the 3D model that track with the camera movement.



Submission Guidelines

- Submit a link to your public GitHub repository containing the complete source code, 3D assets (in a public folder), and Dockerfile (if implemented).
- Ensure your app can be run locally with `npm install` & `npm run dev` (or `npm start`) and built for production.
- Host your app on a free platform like Vercel, Netlify, or GitHub Pages and submit the live URL. (Note: Ensure your 3D models are not too large so they don't break hosting limits).
- If Docker is implemented, include clear instructions in the README for building and running the container.

Backend Track: Authenticated Backend Server with Docker

♦ Task

Design and develop a robust backend server using a stack of your choice—**Go**, **Node.js (Express)**, **Django**, **Flask**, or any other backend technology you're comfortable with.

The server should support **user authentication with JWT**, integrate with a database (MongoDB/PostgreSQL), and be fully **containerized using Docker Compose**.

Objectives

Build a production-ready backend service that showcases secure authentication, clean code structure, and containerized deployment.

Your implementation will be evaluated on correctness, code clarity, security practices (e.g., password hashing, JWT usage), and Docker setup.

Core Features

Authentication Routes

POST **/signup**

- Accepts **username** and **password** from the user.
- Securely hash the password before storing it in the database.
- Return a success response upon registration.

GET **/signin**

- Authenticate the user by verifying credentials.
- Return a signed **JWT token** if credentials are valid.

Protected User Route

GET **/me**

- Requires a valid JWT in the Authorization header.
- Returns the authenticated user's `username` and `description`.

PUT `/me`

- Requires a valid JWT.
- Accepts a new `description` and updates it for the logged-in user.

Database Integration

Use **MongoDB** or **PostgreSQL** to store user data.

User Schema must include:

- `username` (string, unique)
- `hashed_password` (string, securely hashed)
- `description` (string, editable by the user)

You can use any ORM/ODM (e.g., Mongoose for MongoDB, SQLAlchemy/Prisma/TypeORM for relational DBs).

Dockerization

Use **Docker Compose** to manage multi-container setup including:

- Backend service
- Database service

Your Compose setup should include:

- `.env` support for DB credentials and JWT secrets
- Proper networking between backend and DB services

- Health checks or [wait-for-it](#) scripts to ensure DB is ready before backend starts

Bonus Points

Extra Features (Optional but Impressive)

- **Rate Limiting:** Prevent brute-force or spammy login attempts.
- **Logging:** Implement request or error logging (e.g., using [winston](#), [Loguru](#), or [zap](#)).
- **Input Validation:** Prevent malformed requests using validation libraries like [Joi](#), [Pydantic](#), or [express-validator](#).
- **Persistent Volumes:** Use Docker volumes for retaining database data between container restarts.
- **Testing:** Add basic unit or integration tests for routes.

Submission Guidelines

- Submit a link to your **public GitHub repository** with the full source code and a Docker Compose setup.
- The backend should be runnable locally with:
- Include a **README.md** with:
 - Setup and usage instructions
 - API endpoints overview
 - Optional enhancements or known limitations

Blockchain Track

♦ Level 1: Smart Contract Development

Create and deploy an **ERC-20 token** with advanced capabilities.

Required Features

- **Capped Supply** – Limit the total supply.
- **Burnable** – Allow users to destroy tokens.
- **Minting with Rewards** – Enable minting with optional rewards (avoid transaction loops).
- **Self-Destruct** – Implement a secure `selfDestruct()` (with proper access control).

Deploy the contract on a testnet (e.g., Sepolia, Base Sepolia, Polygon Amoy) and ensure it's verifiable on a block explorer.

♦ Level 2: Frontend Integration

Build a frontend (React or any framework/library) to interact with the contract. Include:

- **MetaMask connection**
- Display user **token balance**
- Provide buttons/UI for:
 - Minting
 - Burning
 - Transferring
 - Self-destruct (if applicable)

 Your goal is to **replicate Remix IDE interactions** in a user-friendly interface.

Faucets for Testnets:- [ChainLink](#), [Google Cloud](#)



Submission Instructions

1. GitHub Repository

- Create a public GitHub repository for your project.
- Include a **clear and well-structured README** with:
 - Project Overview
 - Features Implemented
 - Technologies Used
 - Setup & Build Instructions
 - Screenshots / Demo
 - Any Known Issues or Future Work

2. Code Structure

- Organize your code properly into folders.
- Include comments where necessary.
- Dockerfiles / docker-compose.yml should be in the root or a clearly marked directory.

3. Working Build

- Your project should be **runnable locally** using the provided instructions.
- If you've used Docker, ensure the Dockerfile/Compose setup works as expected.



Repo Checklist:

- README with setup steps and project explanation
- Dockerfile or docker-compose.yml (if required by your track)
- Working code for selected task

- Bonus features or improvements (if any)
- Screenshots / Recordings (optional but appreciated)

Submission Link:

The submission form link will be shared **soon**.

If you have any doubts, feel free to reach out on our WhatsApp group.

Best of luck — we're excited to see what you build!

Resources

- [Docker – Full Course](#)
- [React Full Playlist](#)
- [Go Lang Full Course – FreeCodeCamp](#)
- [Node & Express Full Tutorial](#)
- [Python Full Course – FreeCodeCamp](#)
- [Tailwind CSS Crash Course](#)
- [MongoDB Tutorial for Beginners](#)
- [PostgreSQL Full Course](#)
- [Blockchain Development Tutorial \(Patrick Collins\)](#)